

3.2 程序编码

假设一个 C 程序，有两个文件 `p1.c` 和 `p2.c`。我们用 Unix 命令行编译这些代码：

```
linux> gcc -Og -o p p1.c p2.c
```

命令 `gcc` 指的就是 GCC C 编译器。因为这是 Linux 上默认的编译器，我们也可以简单地用 `cc` 来启动它。编译选项 `-Og`^① 告诉编译器使用会生成符合原始 C 代码整体结构的机器代码的优化等级。使用较高级别优化产生的代码会严重变形，以至于产生的机器代码和初始源代码之间的关系非常难以理解。因此我们会使用 `-Og` 优化作为学习工具，然后当我们增加优化级别时，再看会发生什么。实际中，从得到的程序的性能考虑，较高级别的优化(例如，以选项 `-O1` 或 `-O2` 指定)被认为是较好的选择。

实际上 `gcc` 命令调用了一整套的程序，将源代码转化成可执行代码。首先，C 预处理器扩展源代码，插入所有用 `#include` 命令指定的文件，并扩展所有用 `#define` 声明指定的宏。其次，编译器产生两个源文件的汇编代码，名字分别为 `p1.s` 和 `p2.s`。接下来，汇编器会将汇编代码转化成二进制目标代码文件 `p1.o` 和 `p2.o`。目标代码是机器代码的一种形式，它包含所有指令的二进制表示，但是还没有填入全局值的地址。最后，链接器将两个目标代码文件与实现库函数(例如 `printf`)的代码合并，并产生最终的可执行代码文件 `p` (由命令行指示符 `-o p` 指定的)。可执行代码是我们要考虑的机器代码的第二种形式，也就是处理器执行的代码格式。我们会在第 7 章更详细地介绍这些不同形式的机器代码之间的关系以及链接的过程。

3.2.1 机器级代码

正如在 1.9.3 节中讲过的那样，计算机系统使用了多种不同形式的抽象，利用更简单的抽象模型来隐藏实现的细节。对于机器级编程来说，其中两种抽象尤为重要。第一种是由指令集体系结构或指令集架构(Instruction Set Architecture, ISA)来定义机器级程序的格式和行为，它定义了处理器状态、指令的格式，以及每条指令对状态的影响。大多数 ISA，包括 `x86-64`，将程序的行为描述成好像每条指令都是按顺序执行的，一条指令结束后，下一条再开始。处理器的硬件远比描述的精细复杂，它们并发地执行许多指令，但是可以采取保证整体行为与 ISA 指定的顺序执行的行为完全一致。第二种抽象是，机器级程序使用的内存地址是虚拟地址，提供的内存模型看上去是一个非常大的字节数组。存储器系统的实际实现是将多个硬件存储器和操作系统软件组合起来，这会在第 9 章中讲到。

在整个编译过程中，编译器会完成大部分的工作，将把用 C 语言提供的相对比较抽象的执行模型表示的程序转化成处理器执行的非常基本的指令。汇编代码表示非常接近于机器代码。与机器代码的二进制格式相比，汇编代码的主要特点是它用可读性更好的文本格式表示。能够理解汇编代码以及它与原始 C 代码的联系，是理解计算机如何执行程序的关键一步。

`x86-64` 的机器代码和原始的 C 代码差别非常大。一些通常对 C 语言程序员隐藏的处理器状态都是可见的：

- 程序计数器(通常称为“PC”，在 `x86-64` 中用 `%rip` 表示)给出将要执行的下一条指令在内存中的地址。

① GCC 版本 4.8 引入了这个优化等级。较早的 GCC 版本和其他一些非 GNU 编译器不认识这个选项。对这样一些编译器，使用一级优化(由命令行标志 `-O1` 指定)可能是最好的选择，生成的代码能够符合原始程序的结构。

- 整数寄存器文件包含 16 个命名的位置，分别存储 64 位的值。这些寄存器可以存储地址（对应于 C 语言的指针）或整数数据。有的寄存器被用来记录某些重要的程序状态，而其他的寄存器用来保存临时数据，例如过程的参数和局部变量，以及函数的返回值。
- 条件码寄存器保存着最近执行的算术或逻辑指令的状态信息。它们用来实现控制或数据流中的条件变化，比如说用来实现 if 和 while 语句。
- 一组向量寄存器可以存放一个或多个整数或浮点数值。

虽然 C 语言提供了一种模型，可以在内存中声明和分配各种数据类型的对象，但是机器代码只是简单地将内存看成一个很大的、按字节寻址的数组。C 语言中的聚合数据类型，例如数组和结构，在机器代码中用一组连续的字节来表示。即使是对标量数据类型，汇编代码也不区分有符号或无符号整数，不区分各种类型的指针，甚至于不区分指针和整数。

程序内存包含：程序的可执行机器代码，操作系统需要的一些信息，用来管理过程调用和返回的运行时栈，以及用户分配的内存块（比如说用 malloc 库函数分配的）。正如前面提到的，程序内存用虚拟地址来寻址。在任意给定的时刻，只有有限的一部分虚拟地址被认为是合法的。例如，x86-64 的虚拟地址是由 64 位的字来表示的。在目前的实现中，这些地址的高 16 位必须设置为 0，所以一个地址实际上能够指定的是 2^{48} 或 64TB 范围内的一个字节。较为典型的程序只会访问几兆字节或几千兆字节的数据。操作系统负责管理虚拟地址空间，将虚拟地址翻译成实际处理器内存中的物理地址。

一条机器指令只执行一个非常基本的操作。例如，将存放在寄存器中的两个数字相加，在存储器和寄存器之间传送数据，或是条件分支转移到新的指令地址。编译器必须产生这些指令的序列，从而实现（像算术表达式求值、循环或过程调用和返回这样的）程序结构。

旁注 不断变化的生成代码的格式

在本书的表述中，我们给出的代码是由特定版本的 GCC 在特定的命令行选项设置下产生的。如果你在自己的机器上编译代码，很有可能用到其他的编译器或者不同版本的 GCC，因而会产生不同的代码。支持 GCC 的开源社区一直在修改代码产生器，试图根据微处理器制造商提供的不断变化的代码规则，产生更有效的代码。

本书示例的目标是展示如何查看汇编代码，并将它反向映射到高级编程语言中的结构。你需要将这些技术应用到你的特定的编译器产生的代码格式上。

3.2.2 代码示例

假设我们写了一个 C 语言代码文件 mstore.c，包含如下的函数定义：

```
long mult2(long, long);

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}
```

在命令行上使用“-S”选项，就能看到 C 语言编译器产生的汇编代码：

```
linux> gcc -Og -S mstore.c
```

这会使 GCC 运行编译器，产生一个汇编文件 mstore.s，但是不做其他进一步的工作。（通常情况下，它还会继续调用汇编器产生目标代码文件。）

汇编代码文件包含各种声明，包括下面几行：

```
multstore:
    pushq    %rbx
    movq     %rdx, %rbx
    call     mult2
    movq     %rax, (%rbx)
    popq     %rbx
    ret
```

上面代码中每个缩进去的行都对应于一条机器指令。比如，`pushq` 指令表示应该将寄存器 `%rbx` 的内容压入程序栈中。这段代码中已经除去了所有关于局部变量名或数据类型的信息。

如果我们使用 “-c” 命令行选项，GCC 会编译并汇编该代码：

```
linux> gcc -Og -c mstore.c
```

这就会产生目标代码文件 `mstore.o`，它是二进制格式的，所以无法直接查看。1368 字节的文件 `mstore.o` 中有一段 14 字节的序列，它的十六进制表示为：

```
53 48 89 d3 e8 00 00 00 00 48 89 03 5b c3
```

这就是上面列出的汇编指令对应的目标代码。从中得到一个重要信息，即机器执行的程序只是一个字节序列，它是对一系列指令的编码。机器对产生这些指令的源代码几乎一无所知。

旁注 如何展示程序的字节表示

要展示程序(比如说 `mstore`)的二进制目标代码，我们用反汇编器(后面会讲到)确定该过程的代码长度是 14 字节。然后，在文件 `mstore.o` 上运行 GNU 调试工具 GDB，输入命令：

```
(gdb) x/14xb multstore
```

这条命令告诉 GDB 显示(简称为 ‘x’)从函数 `multstore` 所处地址开始的 14 个十六进制格式表示(也简称为 ‘x’)的字节(简称为 ‘b’)。你会发现，GDB 有很多有用的特性可以用来分析机器级程序，我们会在 3.10.2 节中讨论。

要查看机器代码文件的内容，有一类称为反汇编器(disassembler)的程序非常有用。这些程序根据机器代码产生一种类似于汇编代码的格式。在 Linux 系统中，带 ‘-d’ 命令行标志的程序 `OBJDUMP`(表示 “object dump”)可以充当这个角色：

```
linux> objdump -d mstore.o
```

结果如下(这里，我们在左边增加了行号，在右边增加了斜体表示的注解)：

```
Disassembly of function multstore in binary file mstore.o
1  0000000000000000 <multstore>:
   Offset  Bytes                               Equivalent assembly language
2      0:  53                                     push    %rbx
3      1:  48 89 d3                               mov     %rdx,%rbx
4      4:  e8 00 00 00 00                       callq   9 <multstore+0x9>
5      9:  48 89 03                               mov     %rax,(<rbx>)
6      c:  5b                                     pop     %rbx
7      d:  c3                                     retq
```

在左边，我们看到按照前面给出的字节顺序排列的 14 个十六进制字节值，它们分成了若干组，每组有 1~5 个字节。每组都是一条指令，右边是等价的汇编语言。

其中一些关于机器代码和它的反汇编表示的特性值得注意：

- x86-64 的指令长度从 1 到 15 个字节不等。常用的指令以及操作数较少的指令所需的字节数少，而那些不太常用或操作数较多的指令所需字节数较多。
- 设计指令格式的方式是，从某个给定位置开始，可以将字节唯一地解码成机器指令。例如，只有指令 `pushq %rbx` 是以字节值 53 开头的。
- 反汇编器只是基于机器代码文件中的字节序列来确定汇编代码。它不需要访问该程序的源代码或汇编代码。
- 反汇编器使用的指令命名规则与 GCC 生成的汇编代码使用的有些细微的差别。在我们的示例中，它省略了很多指令结尾的‘q’。这些后缀是大小指示符，在大多数情况下可以省略。相反，反汇编器给 `call` 和 `ret` 指令添加了‘q’后缀，同样，省略这些后缀也没有问题。

生成实际可执行的代码需要对一组目标代码文件运行链接器，而这一组目标代码文件中必须含有一个 `main` 函数。假设在文件 `main.c` 中有下面这样的函数：

```
#include <stdio.h>

void multstore(long, long, long *);

int main() {
    long d;
    multstore(2, 3, &d);
    printf("2 * 3 --> %ld\n", d);
    return 0;
}

long mult2(long a, long b) {
    long s = a * b;
    return s;
}
```

然后，我们用如下方法生成可执行文件 `prog`：

```
linux> gcc -Og -o prog main.c mstore.c
```

文件 `prog` 变成了 8 655 个字节，因为它不仅包含了两个过程的代码，还包含了用来启动和终止程序的代码，以及用来与操作系统交互的代码。我们也可以反汇编 `prog` 文件：

```
linux> objdump -d prog
```

反汇编器会抽取各种代码序列，包括下面这段：

```
Disassembly of function sum multstore binary file prog
1  000000000400540 <multstore>:
2      400540: 53                push    %rbx
3      400541: 48 89 d3          mov     %rdx,%rbx
4      400544: e8 42 00 00 00    callq  40058b <mult2>
5      400549: 48 89 03          mov     %rax,(%rbx)
6      40054c: 5b                pop     %rbx
7      40054d: c3                retq
8      40054e: 90                nop
9      40054f: 90                nop
```

这段代码与 `mstore.c` 反汇编产生的代码几乎完全一样。其中一个主要的区别是左边

列出的地址不同——链接器将这段代码的地址移到了一段不同的地址范围中。第二个不同之处在于链接器填上了 `callq` 指令调用函数 `mult2` 需要使用的地址(反汇编代码第4行)。链接器的任务之一就是为函数调用找到匹配的函数的可执行代码的位置。最后一个区别是多了两行代码(第8和9行)。这两条指令对程序没有影响,因为它们出现在返回指令后面(第7行)。插入这些指令是为了使函数代码变为16字节,使得就存储器系统性能而言,能更好地放置下一个代码块。

3.2.3 关于格式的注解

GCC 产生的汇编代码对我们来说有点儿难读。一方面,它包含一些我们不需要关心的信息,另一方面,它不提供任何程序的描述或它是如何工作的描述。例如,假设我们用如下命令生成文件 `mstore.s`。

```
linux> gcc -Og -S mstore.c
```

`mstore.s` 的完整内容如下:

```
.file "010-mstore.c"
.text
.globl multstore
.type multstore, @function
multstore:
    pushq    %rbx
    movq     %rdx, %rbx
    call     mult2
    movq     %rax, (%rbx)
    popq     %rbx
    ret
.size      multstore, .-multstore
.ident     "GCC: (Ubuntu 4.8.1-2ubuntu1~12.04) 4.8.1"
.section   .note.GNU-stack,"",@progbits
```

所有以‘.’开头的行都是指导汇编器和链接器工作的伪指令。我们通常可以忽略这些行。另一方面,也没有关于指令的用途以及它们与源代码之间关系的解释说明。

为了更清楚地说明汇编代码,我们用这样一种格式来表示汇编代码,它省略了大部分伪指令,但包括行号和解释性说明。对于我们的示例,带解释的汇编代码如下:

```
void multstore(long x, long y, long *dest)
x in %rdi, y in %rsi, dest in %rdx
1  multstore:
2  pushq    %rbx           Save %rbx
3  movq     %rdx, %rbx     Copy dest to %rbx
4  call     mult2          Call mult2(x, y)
5  movq     %rax, (%rbx)   Store result at *dest
6  popq     %rbx           Restore %rbx
7  ret                     Return
```

通常我们只会给出与讨论内容相关的代码行。每一行的左边都有编号供引用,右边是注释,简单地描述指令的效果以及它与原始 C 语言代码中的计算操作的关系。这是一种汇编语言程序员写代码的风格。

我们还提供网络旁注,为专门的机器语言爱好者提供一些资料。一个网络旁注描述的是 IA32 机器代码。有了 x86-64 的背景,学习 IA32 会相当简单。另外一个网络旁注简要

描述了在 C 语言中插入汇编代码的方法。对于一些应用程序，程序员必须用汇编代码来访问机器的低级特性。一种方法是用汇编代码编写整个函数，在链接阶段把它们和 C 函数组合起来。另一种方法是利用 GCC 的支持，直接在 C 程序中嵌入汇编代码。

旁注 ATT 与 Intel 汇编代码格式

我们的表述是 ATT(根据“AT&T”命名的，AT&T 是运营贝尔实验室多年的公司)格式的汇编代码，这是 GCC、OBJDUMP 和其他一些我们使用的工具的默认格式。其他一些编程工具，包括 Microsoft 的工具，以及来自 Intel 的文档，其汇编代码都是 Intel 格式的。这两种格式在许多方面有所不同。例如，使用下述命令行，GCC 可以产生 multstore 函数的 Intel 格式的代码：

```
linux> gcc -Og -S -masm=intel mstore.c
```

这个命令得到下列汇编代码：

```
multstore:
    push    rbx
    mov     rbx, rdx
    call    mult2
    mov     QWORD PTR [rbx], rax
    pop     rbx
    ret
```

我们看到 Intel 和 ATT 格式在如下方面有所不同：

- Intel 代码省略了指示大小的后缀。我们看到指令 push 和 mov，而不是 pushq 和 movq。
- Intel 代码省略了寄存器名字前面的‘%’符号，用的是 rbx，而不是 %rbx。
- Intel 代码用不同的方式来描述内存中的位置，例如是‘QWORD PTR [rbx]’而不是‘(%rbx)’。
- 在带有多个操作数的指令情况下，列出操作数的顺序相反。当在两种格式之间进行转换的时候，这一点非常令人困惑。

虽然在我们的表述中不使用 Intel 格式，但是在来自 Intel 和 Microsoft 的文档中，你会遇到它。

网络旁注 ASM:EASM 把 C 程序和汇编代码结合起来

虽然 C 编译器在把程序中表达的计算转换到机器代码方面表现出色，但是仍然有一些机器特性是 C 程序访问不到的。例如，每次 x86-64 处理器执行算术或逻辑运算时，如果得到的运算结果的低 8 位中有偶数个 1，那么就会把一个名为 PF 的 1 位条件码(condition code)标志设置为 1，否则就设置为 0。这里的 PF 表示“parity flag(奇偶标志)”。在 C 语言中计算这个信息需要至少 7 次移位、掩码和异或运算(参见习题 2.65)。即使作为每次算术或逻辑运算的一部分，硬件都完成了这项计算，而 C 程序却无法知道 PF 条件码标志的值。在程序中插入几条汇编代码指令就能很容易地完成这项任务。

在 C 程序中插入汇编代码有两种方法。第一种是，我们可以编写完整的函数，放进一个独立的汇编代码文件中，让汇编器和链接器把它和用 C 语言书写的代码合并起来。第二种方法是，我们可以使用 GCC 的内联汇编(inline assembly)特性，用 asm 伪指令可以在 C 程序中包含简短的汇编代码。这种方法的好处是减少了与机器相关的代码量。

当然，在 C 程序中包含汇编代码使得这些代码与某类特殊的机器相关(例如 x86-64)，所以只应该在想要的特性只能以此种方式才能访问到时才使用它。

3.3 数据格式

由于是从 16 位体系结构扩展成 32 位的，Intel 用术语“字(word)”表示 16 位数据类型。因此，称 32 位数为“双字(double words)”，称 64 位数为“四字(quad words)”。图 3-1 给出了 C 语言基本数据类型对应的 x86-64 表示。标准 int 值存储为双字(32 位)。指针(在此用 char * 表示)存储为 8 字节的四字，64 位机器本来就预期如此。x86-64 中，数据类型 long 实现为 64 位，允许表示的值范围较大。本章代码示例中的大部分都使用了指针和 long 数据类型，所以都是四字操作。x86-64 指令集同样包括完整的针对字节、字和双字的指令。

C 声明	Intel 数据类型	汇编代码后缀	大小(字节)
char	字节	b	1
short	字	w	2
int	双字	l	4
long	四字	q	8
char*	四字	q	8
float	单精度	s	4
double	双精度	l	8

图 3-1 C 语言数据类型在 x86-64 中的大小。在 64 位机器中，指针长 8 字节

浮点数主要有两种形式：单精度(4 字节)值，对应于 C 语言数据类型 float；双精度(8 字节)值，对应于 C 语言数据类型 double。x86 家族的微处理器历史上实现过对一种特殊的 80 位(10 字节)浮点格式进行全套的浮点运算(参见家庭作业 2.86)。可以在 C 程序中使用声明 long double 来指定这种格式。不过我们不建议使用这种格式。它不能移植到其他类型的机器上，而且实现的硬件也不如单精度和双精度算术运算的高效。

如图所示，大多数 GCC 生成的汇编代码指令都有一个字符的后缀，表明操作数的大小。例如，数据传送指令有四个变种：movb(传送字节)、movw(传送字)、movl(传送双字)和 movq(传送四字)。后缀‘l’用来表示双字，因为 32 位数被看成是“长字(long word)”。注意，汇编代码也使用后缀‘l’来表示 4 字节整数和 8 字节双精度浮点数。这不会产生歧义，因为浮点数使用的是一组完全不同的指令和寄存器。

3.4 访问信息

一个 x86-64 的中央处理单元(CPU)包含一组 16 个存储 64 位值的通用目的寄存器。这些寄存器用来存储整数数据和指针。图 3-2 显示了这 16 个寄存器。它们的名字都以 %r 开头，不过后面还跟着一些不同的命名规则的名字，这是由于指令集历史演化造成的。最初的 8086 中有 8 个 16 位的寄存器，即图 3-2 中的 %ax 到 %bp。每个寄存器都有特殊的用途，它们的名字就反映了这些不同的用途。扩展到 IA32 架构时，这些寄存器也扩展成 32 位寄存器，标号从 %eax 到 %ebp。扩展到 x86-64 后，原来的 8 个寄存器扩展成 64 位，标号从 %rax 到 %rbp。除此之外，还增加了 8 个新的寄存器，它们的标号是按照新的命名规则制定的：从 %r8 到 %r15。